

Del experimento al modelo desplegable

Guía paso a paso con MLflow y Weights & Biases

DVC como complemento de versionado · Caso Wine Quality

Universidad de Cuenca · Asignatura: Ciencia de Datos

Equipo del proyecto: Bryan Macas y Joseph Sangurima

Material educativo · 6 de octubre de 2026

1. Presentación y propósito de la guía

La guía explica cómo se prepara un experimento, se comparan sus resultados y se utiliza el modelo elegido para generar predicciones. El caso práctico corresponde al proyecto *mlops-wine-quality*. Esta versión incorpora la ejecución completa documentada en `informe/flujo_experimento.md`. Se requieren conocimientos básicos de Python y tablas de datos.

MLOps es un conjunto de prácticas para organizar el desarrollo y uso de modelos de aprendizaje automático. Su propósito es conservar los datos, el código, los resultados y el modelo de cada prueba. De esta forma, el equipo puede repetir un experimento y entender de dónde salió una predicción. El **despliegue** permite que otra aplicación envíe datos al modelo y reciba una respuesta.

MLflow, herramienta principal, organiza ejecuciones o *runs*, parámetros, métricas y artefactos. Nació en Databricks y se incorporó a la Linux Foundation en 2020; su comunidad desarrolla el proyecto abierto bajo Apache 2.0 (Linux Foundation, 2020; MLflow, s. f.-b). No exige pago de licencia para utilizar el código abierto, pero operar servidores, almacenamiento y respaldos cuesta recursos.

Weights & Biases (W&B) es la alternativa implementada: centraliza experimentos y versiones de artefactos. Su SDK Python usa licencia MIT; esto no convierte al servicio alojado en un producto sin restricciones comerciales. W&B forma parte de CoreWeave y ofrece modalidades gratuitas y pagadas, incluida una modalidad académica sujeta a condiciones (Weights & Biases, s. f.-a, s. f.-b).

DVC complementa Git con versiones de datos, modelos y pipelines; el proyecto fue impulsado por Iterative y su repositorio oficial consultado redirige a `treeverse/dvc`. Tiene licencia Apache 2.0 y puede utilizar almacenamiento propio. Se estudia como alternativa conceptual: **este repositorio no lo implementa ni lo mide** (DVC, s. f.-a, s. f.-b).

Cuándo elegir cada enfoque

MLflow resulta útil para registrar experimentos y guardar modelos en un equipo local. W&B facilita la comparación y colaboración mediante una plataforma compartida, aunque también permite guardar ejecuciones sin conexión. DVC ayuda a conservar versiones de archivos grandes y a repetir las etapas de un proyecto junto con Git. Las tres herramientas pueden complementarse.

Alcance comprobado. La ejecución del 6 de octubre de 2026 completó **63 celdas de código sin errores**, incluida una celda adicional de verificación. Se finalizaron **18 runs de MLflow** y se generaron **17 ejecuciones de W&B offline**. El modelo se recargó y respondió correctamente a una petición HTTP local. W&B no se sincronizó con la nube y DVC no se ejecutó.

Recorrido de aprendizaje: preparación del entorno, carga y limpieza de datos, entrenamiento, comparación, evaluación final y prueba local de predicción. Los comandos muestran cómo se reproduce el trabajo; las tablas y figuras presentan resultados de la ejecución documentada.

2. Ubicación en el ciclo de vida e instalación

Etapa	Trabajo del proyecto	Aporte MLOps
Recopilación	Obtener Wine Quality desde UCI.	Conservar fuente e identificador.
Limpieza	Revisar faltantes y retirar duplicados.	Exportar datos y metadatos.
Modelado	Entrenar Random Forest con diez configuraciones.	Registrar parámetros y semilla por run.
Evaluación	Seleccionar por validación; evaluar en prueba aislada.	Separar métricas de selección y finales.
Visualización	Comparar RMSE, R^2 y tiempos.	Consultar CSV y paneles de runs.
Despliegue	Cargar el modelo y probar una petición HTTP local.	Comprobar que el modelo guardado produce las mismas predicciones.

El flujo comprobado llega hasta una prueba de predicción en el propio equipo. Un servicio de producción requeriría controles adicionales de acceso, disponibilidad y seguimiento de errores.

Paso 1. Preparar un entorno aislado

La preparación comienza en la raíz del proyecto. Los dos primeros comandos solo corresponden a una copia nueva; la ejecución documentada partió de la copia existente y creó su entorno virtual con los comandos restantes:

```
git clone https://github.com/bryzcoll/mlops-wine-quality.git
cd mlops-wine-quality
py -3.12 -m venv .venv
.\venv\Scripts\python.exe -m pip install -r requirements.txt
.\venv\Scripts\python.exe -m pip check
```

El entorno virtual separa las bibliotecas del proyecto de las utilizadas por otros programas. La instalación terminó correctamente y `pip check` devolvió `No broken requirements found`, es decir, no detectó dependencias incompatibles.

Entorno de esta ejecución: Windows 11, Python 3.12.10, MLflow 3.16.1, W&B 0.30.0, scikit-learn 1.9.1, pandas 3.0.6 y NumPy 2.5.3. El equipo tenía un Intel Core i7-8565U, 4 núcleos, 8 procesadores lógicos y aproximadamente 19.82 GiB de memoria. Las versiones completas quedaron en `informe/ejecucion/requirements_resueltos.txt`.

Paso 2. Ejecutar el notebook y conservar la evidencia

```
$env:WANDB_MODE="offline"
.\venv\Scripts\python.exe informe\ejecutar_flujo.py
```

El programa ejecuta una copia del notebook, celda por celda, con el Python del entorno virtual. Los resultados nuevos se guardan en `informe/ejecucion`; los archivos originales se conservan. El notebook tardó **200.11 s**. El tiempo acumulado, incluidos los intentos de inferencia, fue **222.77 s**, sin contar instalación ni diagnóstico.

La descarga de UCI requiere Internet. El modo *offline* de W&B guarda resultados localmente y no requiere iniciar sesión. Si el programa se ejecuta otra vez, sustituye los CSV de esta carpeta y añade nuevos runs; por ello, la conservación de una corrida completa requiere un respaldo previo.

3. Caso de uso: datos y preparación

Paso 3. Definir la pregunta y cargar los datos

El problema es estimar la puntuación sensorial `quality` a partir de once medidas fisicoquímicas. Se modela como regresión: una predicción puede ser decimal aunque la etiqueta sea entera. El conjunto UCI Wine Quality reúne muestras de vinos tintos y blancos portugueses y está disponible bajo CC BY 4.0 (Cortez et al., 2009). La licencia del repositorio del grupo es MIT; no sustituye la licencia del dataset.

```
from ucimlrepo import fetch_ucirepo
import pandas as pd
wine = fetch_ucirepo(id=186)
df = pd.concat([wine.data.features, wine.data.targets], axis=1)
print(df.shape, df.isna().sum().sum())
df = df.drop_duplicates().reset_index(drop=True)
X, y = df.drop(columns="quality"), df["quality"]
```

Salida comprobada: 6 497 filas y 12 columnas antes de la limpieza; cero datos faltantes; 1 179 filas duplicadas eliminadas; 5 318 filas finales. Las once entradas incluyen acidez fija y volátil, ácido cítrico, azúcar residual, cloruros, dióxido de azufre libre y total, densidad, pH, sulfatos y alcohol. El modelo no utiliza una variable explícita de tipo de vino.

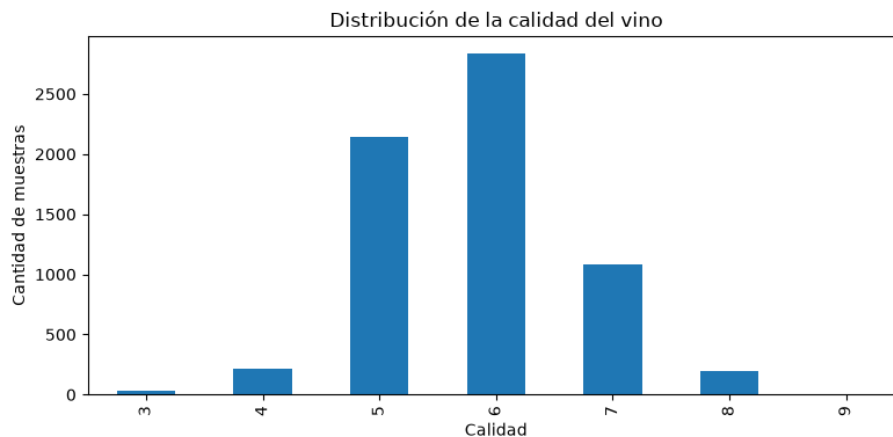


Figura 1: Distribución obtenida en la nueva ejecución, antes de eliminar duplicados. Las puntuaciones 5 y 6 son las más frecuentes.

Paso 4. Separar entrenamiento, validación y prueba

```
from sklearn.model_selection import train_test_split
X_dev, X_test, y_dev, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(
    X_dev, y_dev, test_size=0.25, stratify=y_dev, random_state=42)
```

Se obtienen **3 190 / 1 064 / 1 064** filas: aproximadamente 60/20/20. La segunda proporción es 25 % del 80 % restante. La estratificación conserva aproximadamente la distribución de calidad. Retirar duplicados antes de dividir reduce la presencia de filas idénticas en conjuntos distintos, aunque no garantiza independencia por bodega o lote.

Idea clave: el entrenamiento sirve para aprender, la validación para elegir la configuración y la prueba para evaluar el modelo elegido. Si la prueba se utilizara para elegir parámetros, dejaría de ser una evaluación independiente.

4. Seguimiento y trazabilidad paso a paso

Paso 5. Registrar una ejecución con MLflow

Un *run* es el registro de una ejecución. Un *experimento* agrupa esos registros; un *parámetro* describe la configuración; una *métrica* mide el resultado; un *artefacto* es un archivo guardado, como el modelo. El siguiente ejemplo resume el código del notebook; requiere los datos del paso 4 y se ejecuta desde la raíz del proyecto.

```
from pathlib import Path
import numpy as np
import mlflow, mlflow.sklearn
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
db = Path('informe/ejecucion/mlflow.db').resolve().as_posix()
mlflow.set_tracking_uri(f"sqlite:///db")
mlflow.set_experiment("wine-quality-random-forest")
params = dict(n_estimators=300, max_depth=None,
              min_samples_split=5, min_samples_leaf=2, random_state=42)
with mlflow.start_run(run_name="ejemplo-guia"):
    model = RandomForestRegressor(**params, n_jobs=-1)
    model.fit(X_train, y_train)
    pred = model.predict(X_val)
    mlflow.log_params(params)
    mlflow.log_metrics({
        "mae_validacion": mean_absolute_error(y_val, pred),
        "rmse_validacion": float(np.sqrt(mean_squared_error(y_val, pred))),
        "r2_validacion": r2_score(y_val, pred)})
    mlflow.sklearn.log_model(sk_model=model, name="random_forest",
                             skops_trusted_types=["sklearn.tree._tree.Tree"])
```

La consulta opcional de los runs utiliza la base de datos de la ejecución. El siguiente comando abre la interfaz local; no fue necesario ejecutarlo para validar las métricas:

```
.\.venv\Scripts\mlflow.exe ui --backend-store-uri sqlite:///informe/ejecucion/mlflow.db --host 127.0.0.1 --port 5000
```

La interfaz queda disponible en <http://127.0.0.1:5000>. La ordenación por `rmse_validacion`, de menor a mayor, permite comparar las configuraciones. El fragmento anterior crearía un run didáctico adicional; las cifras de esta guía proceden de la ejecución completa del notebook.

Paso 6. Repetir el seguimiento con W&B y guardar artefactos

```
import wandb
with wandb.init(project="wine-quality-guia", config=params,
                mode="offline") as run:
    run.log({"rmse_validacion": float(np.sqrt(mean_squared_error(y_val, pred)))})
```

El fragmento registra una predicción ya calculada; el notebook completo entrena de nuevo las diez configuraciones. La comparación conserva los mismos datos, partición, semilla y parámetros. Los 18 runs de MLflow incluyen baseline, diez configuraciones, prueba final, cinco mediciones de tiempo y un registro de datos. W&B guarda 17 ejecuciones offline: diez configuraciones, cinco mediciones y dos registros de artefactos.

Seguimiento de los archivos: un hash es una huella calculada a partir del contenido de un archivo. El CSV nuevo y su metadato coinciden en el SHA-256 `b22fdcb4...e507a5ab`. La versión original Linux tenía otra huella por sus saltos de línea; al igualarlos, las huellas coinciden. El registro de datos se guarda por separado: asociar su hash a cada entrenamiento facilitaría identificar con qué datos se entrenó cada modelo.

5. Resultados reales y cómo interpretarlos

Paso 7. Elegir la configuración usando validación

La tabla presenta los resultados nuevos de `informe/ejecucion/results/metrics/mlflow_experimentos.csv`. Profundidad “libre” significa que no se fija un límite; división y hoja indican las cantidades mínimas de muestras para dividir un nodo y formar una hoja del árbol.

Exp.	Árboles	Prof.	División	Hoja	RMSE val.	R^2 val.
1	50	5	2	1	0.721668	0.327628
2	100	5	2	1	0.721903	0.327189
3	200	5	2	1	0.722546	0.325990
4	100	10	2	1	0.695421	0.375645
5	200	10	2	1	0.695530	0.375451
6	300	10	2	1	0.694549	0.377210
7	100	libre	2	1	0.691357	0.382922
8	200	libre	5	1	0.690191	0.385001
9	200	libre	2	2	0.689100	0.386945
10	300	libre	5	2	0.688779	0.387516

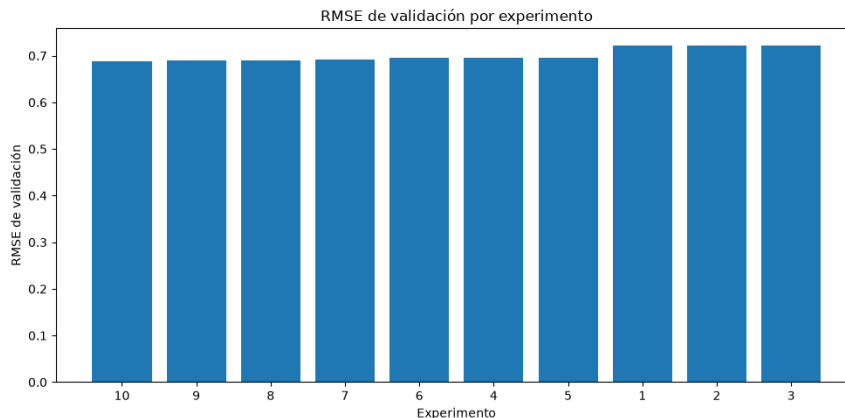


Figura 2: RMSE de validación obtenido en la nueva ejecución. Las barras están ordenadas de menor a mayor error.

El experimento 10 gana por RMSE, aunque el 9 tiene menor MAE (0.534300 frente a 0.535146). La diferencia de RMSE entre ambos es apenas 0.000321: sin intervalos ni otras particiones no demuestra una superioridad robusta. El criterio debe elegirse antes de mirar la prueba.

Paso 8. Reproducir y evaluar una sola vez

La nueva ejecución confirmó **Experimento reproducible: True**: al repetir el entrenamiento, las tres métricas coincidieron dentro de una tolerancia de 10^{-10} . Después se reentrenó la configuración 10 con 4 254 filas y se evaluó sobre las 1 064 filas de prueba.

Métrica	Validación	Prueba final
MAE	0.535146	0.534196
RMSE	0.688779	0.689675
R^2	0.387516	0.385921

El MAE indica un error medio de 0.534 puntos de calidad. El RMSE penaliza más los errores grandes. El R^2 indica que el modelo explica aproximadamente el 38.6 % de la variación de calidad en la prueba; no es un porcentaje de aciertos. Las métricas de MLflow y W&B coinciden, con diferencia máxima de 2.22×10^{-16} . La tabla final procede de `informe/ejecucion/results/metrics/evaluacion_final.csv`.

6. Comparación estructurada de herramientas

Paso 9. Medir el costo del seguimiento

El *benchmark*, o prueba de tiempos, realizó cinco repeticiones por condición con la configuración ganadora. Midió entrenamiento, predicción y cálculo de métricas; con seguimiento, también incluyó el inicio y cierre del run. No guardó modelos durante esta medición. La sobrecarga representa el tiempo adicional respecto a trabajar sin seguimiento:

$$100 \left(\frac{\text{mediana}(t_{\text{tracking}})}{\text{mediana}(t_{\text{sin tracking}})} - 1 \right).$$

Condición	Mediana (s)	Sobrecarga
Sin tracking	1.658416	0 %
MLflow local	1.813136	9.33 %
W&B offline	2.140149	29.05 %

Fuente: `informe/ejecucion/results/metrics/comparacion_sobrecarga.csv`. En esta máquina, el seguimiento añadió aproximadamente 0.155 s con MLflow y 0.482 s con W&B offline. El registro histórico usaba Linux y W&B remoto; sus tiempos no son directamente comparables con esta ejecución en Windows.

Criterio	MLflow	W&B	DVC
Facilidad de uso	Runs locales y panel; 3 métodos básicos contados.	Inicio con configuración y log; 2 métodos contados.	Flujo de archivos y Git; no evaluado en el proyecto.
Curva de aprendizaje	Runs, almacenes y modelos; servidor añade complejidad.	Runs, cuentas y Artifacts; permisos y sincronización.	Git, caché, remotos y dependencias de etapas.
Desempeño medido	1.813136 s; 9.33 % adicional.	2.140149 s; 29.05 % adicional, sin sincronización.	Sin mediciones en el proyecto.
Costo	Código abierto; cómputo, operación y disco propios.	SDK abierto; servicio sujeto a plan y cuotas.	Código abierto; almacenamiento y operación propios.
Comunidad y documentación	Repositorio público; guías de seguimiento y despliegue.	SDK público; guías, soporte y ejemplos del proveedor.	Repositorio público; guías de CLI y pipelines.
Integración con Python	API directa y soporte de scikit-learn.	SDK <code>wandb</code> y Artifacts.	API Python, CLI y DVCLive.
Rol en despliegue	Empaquetado y servidor de inferencia.	Linaje y gestión de artefactos; requiere infraestructura de inferencia.	Recuperación de datos/modelos; requiere servidor de inferencia.

Lectura crítica. La facilidad y la curva de aprendizaje se describen de forma cualitativa. Contar tres métodos frente a dos no mide el tiempo necesario para aprender. Las cinco repeticiones se realizaron en un orden fijo, sin intervalos de confianza; por ello, no permiten declarar una herramienta universalmente más rápida. Tampoco se midió el tamaño de sus comunidades.

Fuentes de capacidades: repositorios y documentación oficiales (MLflow, s. f.-a, s. f.-b; Weights & Biases, s. f.-a, s. f.-b; DVC, s. f.-a, s. f.-b). Para este taller MLflow ofrece una base local; W&B facilita compartir evidencia. DVC sería una extensión cuando los archivos y etapas necesiten versionado ligado a Git.

7. Prueba de despliegue local y errores resueltos

Paso 10. Cargar el modelo y obtener predicciones

La **inferencia** consiste en utilizar un modelo ya entrenado para calcular predicciones sobre datos de entrada. En esta ejecución se cargó el modelo final guardado en MLflow y se enviaron cinco filas al servicio local. No fue necesario volver a entrenarlo para realizar esta prueba.

El modelo pertenece al run `056434b64c66479b8f7b43cfe77ba76e`, con nombre `random_forest_final`. El archivo `informe/ejecucion/evidencia.json` conserva su identificador y las predicciones esperadas. La verificación puede repetirse, desde la raíz, con:

```
.\.venv\Scripts\python.exe informe\verificar_despliegue.py
```

El programa localiza el entorno virtual, inicia el servicio en `127.0.0.1:5001`, comprueba su estado y envía una petición a `POST /invocations`. El mensaje utiliza JSON y conserva el orden de las once variables. Después compara la respuesta con el modelo en memoria y cierra el servidor. El servicio local está descrito en MLflow (s. f.-a).

Comprobación	Resultado real
Estado de la petición HTTP	200 (correcto)
Filas enviadas / predicciones recibidas	5 / 5
Diferencia máxima al recargar el modelo	0
Diferencia máxima entre HTTP y modelo original	0
Tiempo de una petición de cinco filas	0.063266 s

Las predicciones fueron 5.669439, 6.516411, 6.326503, 6.346534 y 5.520499, redondeadas a seis decimales. Los valores completos quedaron en `informe/ejecucion/inferencia_http.json`. El estado 200 confirma la respuesta correcta del servicio; la comparación numérica confirma que utilizó el modelo esperado. Una sola petición no mide capacidad bajo carga.

Errores encontrados y solución aplicada

Incidencia	Solución y resultado
Python no iniciaba en el entorno aislado.	Se utilizó el intérprete fuera de ese aislamiento y se creó el entorno virtual del proyecto.
MLflow no encontraba <code>uvicorn</code> .	Se añadió <code>.venv/Scripts</code> a <code>PATH</code> , la lista de carpetas donde se buscan programas. El reintento HTTP funcionó.
W&B dejó un proceso abierto.	Se cerró el proceso identificado como propio de la ejecución. El ejecutor incorpora cierre explícito para futuras corridas.
El puerto tardó en liberarse.	Una comprobación posterior confirmó que el servidor ya no estaba en escucha.

El primer error HTTP ocurrió después de completar el notebook. Se repitió solo la prueba del servidor, conservando el log fallido. Los cambios finales de cierre de procesos se incorporaron al ejecutor; no se repitió todo el entrenamiento por esos ajustes.

Alcance de la prueba

Se comprobó un **despliegue local de prueba**. Un uso productivo requeriría validar entradas, proteger el acceso, registrar errores, medir carga y vigilar cambios en los datos. También necesitaría respaldar el modelo y disponer de una versión anterior para recuperar el servicio si una actualización falla.

8. Ventajas, límites, riesgos y aplicación externa

Ventajas observadas. Los registros permiten comparar configuraciones y recuperar sus resultados. Los CSV ofrecen un resumen fácil de consultar y los archivos guardados permiten reutilizar el modelo. La nueva ejecución confirmó las métricas anteriores, con una diferencia máxima de 2.22×10^{-16} , atribuible al cálculo numérico. Además, la prueba HTTP mostró que el modelo guardado podía generar las mismas predicciones fuera del entrenamiento.

Riesgo o límite	Efecto en este caso	Medida propuesta
Privacidad y credenciales	Runs pueden incluir rutas, código y archivos; W&B en línea transmite información.	Revisar contenido y permisos antes de sincronizar; no guardar claves en Git.
Sesgo y representatividad	Vinos portugueses y etiquetas sensoriales; predominan calidades intermedias.	Evaluar error por calidad y procedencia; validar externamente antes de generalizar.
Fuga o dependencia de datos	Duplicados eliminados no descartan dependencia por lote.	Partición por grupos cuando haya identificadores; preservar prueba aislada.
Dependencia del proveedor	Acceso remoto, cuotas y planes pueden cambiar.	Exportar métricas y artefactos; documentar recuperación fuera del panel.
Seguridad de modelos	Deserializar archivos no confiables puede ser peligroso.	Usar artefactos propios y verificar origen/hash; no aceptar tipos confiables indiscriminadamente.
Reproducibilidad incompleta	Hash sensible a CRLF; entorno y almacenes no incluidos en Git.	Fijar serialización, versiones y commit; respaldar datos y modelos.

Limitaciones del experimento. Solo se utilizó una partición de datos y se compararon diez configuraciones. Sería útil evaluar otras particiones y comparar con una regla sencilla, como predecir siempre la calidad media. El baseline del notebook es un Random Forest de referencia; en la prueba de tiempos, “baseline” significa trabajar sin seguimiento. Son referencias diferentes.

Límites de las herramientas. En esta corrida, W&B guardó sus archivos localmente; no se comprobó su sincronización ni el panel remoto. DVC se incluyó en la comparación teórica, pero no se instaló ni se midió. Su incorporación permitiría estudiar el versionado de archivos y la repetición de etapas mediante `dvc repro`.

Ejemplo documentado de uso exitoso

Zaharia et al. (2018, p. 43) describen una empresa energética europea que empleaba MLflow para seguir y actualizar cientos de modelos de la red eléctrica. El equipo construía predicciones para productores y consumidores de energía, utilizaba notebooks para desarrollar, MLflow Tracking para métricas y trabajos de Databricks para inferencia.

El resultado documentado es la adopción de un proceso común para manejar muchos modelos y bibliotecas. La publicación no identifica la empresa ni cuantifica ahorro o mejora predictiva; por tanto, no se atribuyen porcentajes de éxito. El caso respalda una aplicación operativa del seguimiento, no una garantía comercial. Para Wine Quality, la lección transferible es mantener un vínculo explícito entre configuración, evaluación y modelo servido.

Actividad de cierre

La actividad propone que el estudiante explique la diferencia entre calidad predictiva y tiempo de seguimiento, justifique una elección de herramienta e identifique qué faltaría para operar en producción. La evidencia mínima incluye el identificador del run, la huella de los datos, las métricas y la ubicación del modelo. Así, otro integrante puede reconocer y revisar el resultado.

9. Referencias (APA 7)

Las fuentes web dinámicas se consultaron el 6 de octubre de 2026. Las letras distinguen obras del mismo autor sin fecha.

- Cortez, P., Cerdeira, A., Almeida, F., Matos, T., & Reis, J. (2009). *Wine quality* [Conjunto de datos]. UCI Machine Learning Repository. <https://doi.org/10.24432/C56S3T>
- DVC. (s. f.-a). *Data versioning and ML experiments* [Software]. GitHub. Recuperado el 6 de octubre de 2026, de <https://github.com/treeverse/dvc>
- DVC. (s. f.-b). *What is DVC?* Recuperado el 6 de octubre de 2026, de <https://doc.dvc.org/user-guide>
- Linux Foundation. (2020, 25 de junio). *The MLflow project joins Linux Foundation*. <https://www.linuxfoundation.org/press/press-release/the-mlflow-project-joins-linux-foundation>
- Macas, B., & Sangurima, J. (2026). *Pipeline MLOps para Wine Quality* [Repositorio de código y resultados]. GitHub. <https://github.com/bryzcoll/mlops-wine-quality>
- MLflow. (s. f.-a). *Deploy MLflow model as a local inference server*. Recuperado el 6 de octubre de 2026, de <https://mlflow.org/docs/latest/ml/deployment/deploy-model-locally/>
- MLflow. (s. f.-b). *MLflow* [Software]. GitHub. Recuperado el 6 de octubre de 2026, de <https://github.com/mlflow/mlflow>
- Weights & Biases. (s. f.-a). *Explore Weights & Biases pricing plans*. Recuperado el 6 de octubre de 2026, de <https://site.wandb.ai/pricing/>
- Weights & Biases. (s. f.-b). *Weights & Biases* [SDK de Python]. GitHub. Recuperado el 6 de octubre de 2026, de <https://github.com/wandb/wandb>
- Zaharia, M., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Konwinski, A., Murching, S., Nykodym, T., Ogilvie, P., Parkhe, M., Xie, F., & Zumar, C. (2018). Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4), 39–45. <https://www.databricks.com/sites/default/files/2023-08/accelerating-the-machine-learning-lifecycle-with-mlflow.pdf>